CS151 Intro to Data Structures

Maps
HashMaps

Warmup: array heap

Draw each array as a tree. Which are valid min-heaps?

- [1, 2, 3, 4, 5]
- [1, 3, 2, 7, 4]
- [2, 5, 3, 9, 6, 8, 4, 10, 7]

Announcements

HW6 due Friday

Lab 8 due Tuesday

Attend Haverford talks and lunches:

- Tomorrow Wed 11th DC (WCC 205 B & C). Talk at 4:15pm KINSC Hilles 109
- Monday Nov 16th DC (WCC 205 B & C). Talk at 4:15pm KINSC Hilles 109
- Wednesday Nov 18th. Talk at 4:15pm KINSC Hilles 109.
- Thursday Nov 19th lunch at Coop.

Grade Reports sent out

Office hours 3-5pm today in Park 205

Priority Queues

Priority Queues

- What is a queue?

- What if we want to create a graduation queue of students who pick up their diplomas in alphabetical order?
  - Queues and Stacks removal is based on when it was added to the data structure
  - Instead we want removal order to be based on the student's name

Priority Queue

A queue that maintains removal order of the elements according to some priority

- generally not related to insertion time (although time of insertion COULD be one criteria)
- each element has an associated priority with it which indicates when it should be removed
- Usually removed based on min priority
- **What data structure can we use to implement a priority queue?**

Priority Queue ADT

insert(k, v): Creates an entry with key k and value v in the priority queue.

min( ): Returns (but does not remove) a priority queue entry (k,v) having minimal key; returns null if the priority queue is empty.

removeMin( ): Removes and returns an entry (k,v) having minimal key from the priority queue; returns null if the priority queue is empty.

size( ): Returns the number of entries in the priority queue.

isEmpty( ): Returns a boolean indicating whether the priority queue is empty.

Implementing a Priority Queue

Method	Unsorted List	Sorted List	Binary Heap
size	$O(1)$	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$	$O(\log n)^*$
min	$O(n)$	$O(1)$	$O(1)$
removeMin	$O(n)$	$O(1)$	$O(\log n)^*$

Priority queue exercise / warmup

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(7);
pq.add(3);
pq.add(9);
System.out.println(pq.poll());
pq.add(1);
pq.add(5);
System.out.println(pq.poll());
System.out.println(pq.peek());
pq.add(4);
System.out.println(pq.poll());
System.out.println(pq.poll());
```

Maps

Map Motivation

Suppose I have an array of Students. What is the runtime complexity of the following operations?

Update the midterm grade of the second student
O(1)

Update the midterm grade for Liam
O(n) I need to search!

Student:

String name;
double[] hwGrades;
double[] labGrades;
double midtermGrade;
...

Student name = Emily	**Student** name = Aiden	**Student** name = Sophia	**Student** name = Liam	**Student** name = Isabella	**Student** name = Noah	**Student** name = Ava ....

Maps

- Also called “dictionaries” or “associative arrays”

- Similar syntax to an array:
  - `m[key]` retrieves a value
  - `m[key] = value` assigns a value
  - keys need not be ints

- data structure that stores a collection of key-value pairs

Key-Value Pairs

- Each element in a map consists of (K, V)

- The key is used to identify the value
  - In an array, this would be the index

- **Examples**: what are the keys and values here?
  - Dictionary
  - Phone Book
  - Student grades

Map

- A indexable collection of key-value pairs
- Multiple entries with the same key are not allowed

Map ADT

- `get(k)`: if the map `M` has an entry with key `k`, return its associated value; else, return null
- `put(k, v)`: insert entry `(k, v)` into the map `M`; if key `k` is not already in M, then return null; else, replace old value with `v` and return old value associated with `k`
- `remove(k)`: if the map `M` has an entry with key `k`, remove it from `M` and return its associated value; else, return null
- `size()`, `isEmpty()`
- `keySet()`: return an iterable collection of the keys in `M`
- `values()`: return an iterator of the values in `M`
- `entrySet()`: return an iterable collection of the entries in `M`

Example

Method	*Return Value*	*Map*
isEmpty()	true	{}
put(5,*A*)	null	{(5,*A*)}

Example

Method	Return Value	Map
isEmpty()	true	{}
put(5,A)	null	{(5,A)}
put(7,B)	null	{(5,A),(7,B)}
put(2,C)	null	{(5,A),(7,B),(2,C)}
put(8,D)	null	{(5,A),(7,B),(2,C),(8,D)}
put(2,E)	C	{(5,A),(7,B),(2,E),(8,D)}
get(7)	B	{(5,A),(7,B),(2,E),(8,D)}
get(4)	null	{(5,A),(7,B),(2,E),(8,D)}
get(2)	E	{(5,A),(7,B),(2,E),(8,D)}
size()	4	{(5,A),(7,B),(2,E),(8,D)}
remove(5)	A	{(7,B),(2,E),(8,D)}
remove(2)	E	{(7,B),(8,D)}
get(2)	null	{(7,B),(8,D)}
remove(2)	null	{(7,B),(8,D)}
isEmpty()	false	{(7,B),(8,D)}
entrySet()	{(7,B),(8,D)}	{(7,B),(8,D)}
keySet()	{7,8}	{(7,B),(8,D)}
values()	{B,D}	{(7,B),(8,D)}

Map ADT

- Map class is abstract
- Concrete Implementations of Map:
  - `UnsortedTableMap`
  - `HashMap`

Map

- How can we implement a map?
  - Array !

Map.Entry Interface

- A (Key, Value) pair
- Keys and Values can be any reference type
- Methods:
  - `getKey()`
  - `getValue()`
  - `setValue(V val)`
- Implementation: `SimpleEntry`

ArrayMap

Let’s implement a `Map` as an array of `SimpleEntry`s

Performance Analysis

	Map implemented with underlying Array	LinkedList
get		
put		
remove		

LinkedList Map

- `get(K key)`
- `put(K key, V value)`
  - If k is not in the map add it. If it is in the map, replace with the new value.
- `remove(K key)`

Performance Analysis

	Array	LinkedList
get	O(n)	
put	O(n)	
remove	O(n)	

Hash Functions

Hash Functions

- A *hash function* maps an arbitrary length input to a fixed length *unique* output
- https://emn178.github.io/online-tools/sha256.html
- Applications
  - data structures
  - encryption / digital signatures
  - blockchain
- Properties of a good hash function:
  - one way
  - **collision resistant**
  - uniformity
  - quick to compute

A Simple Hash Function

Given an int x...

h(x) = last 4 digits of x

- one way?
- collision resistant?
- uniform?

Another Simple Hash Function

h(x) = x % N

- one way?
- collision resistant?
- uniform?

HashMaps

- How can we use hash functions to improve the performance of our ArrayMap implementation?

- **Use hash as our array index!**

- How would the following operations look?
  - put
  - get
  - remove

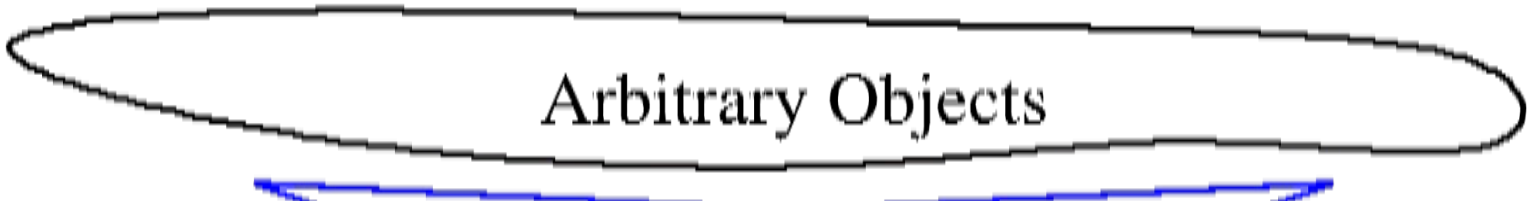
More on Hash Functions

A hash function is usually specified as the composition of two functions:

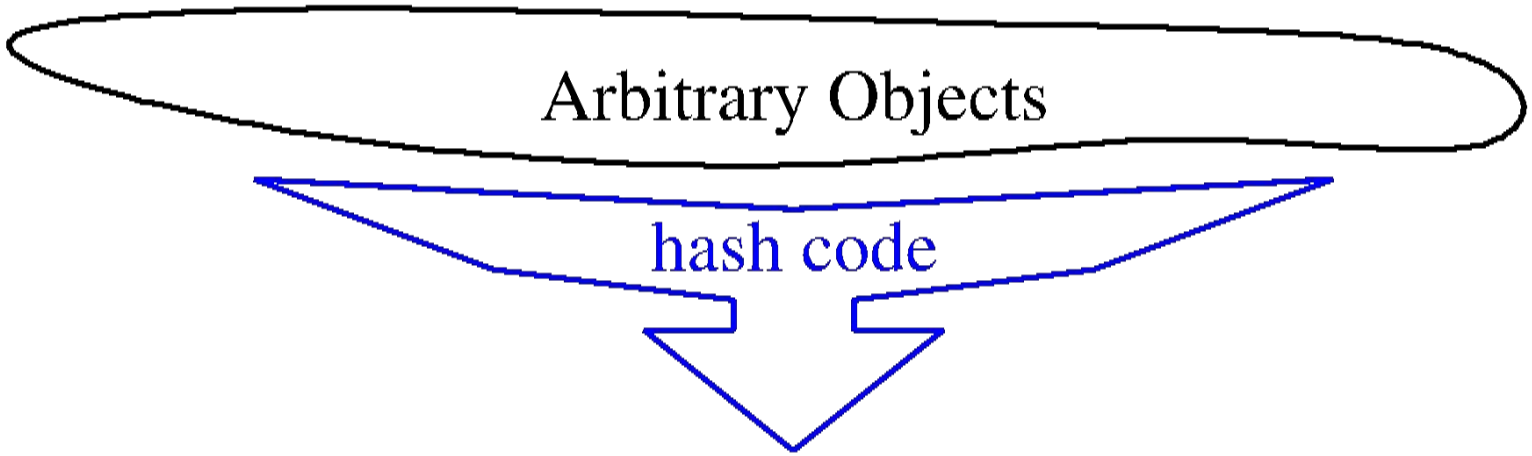
- hash code: h_1: key → integers
- compression: h_2: integers → $[0, N-1]$
- $h(x) = h_2(h_1(x))$

The goal is to "disperse" the keys in an appropriately random way

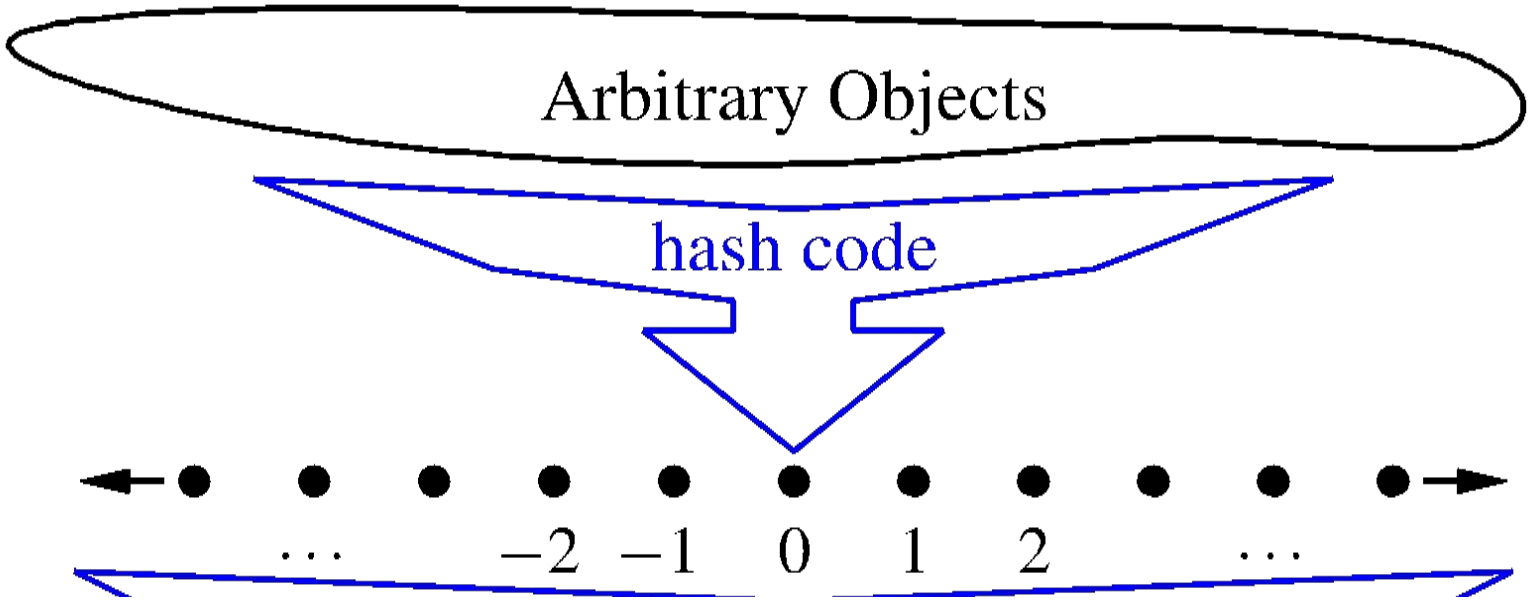
Hash Function Illustration



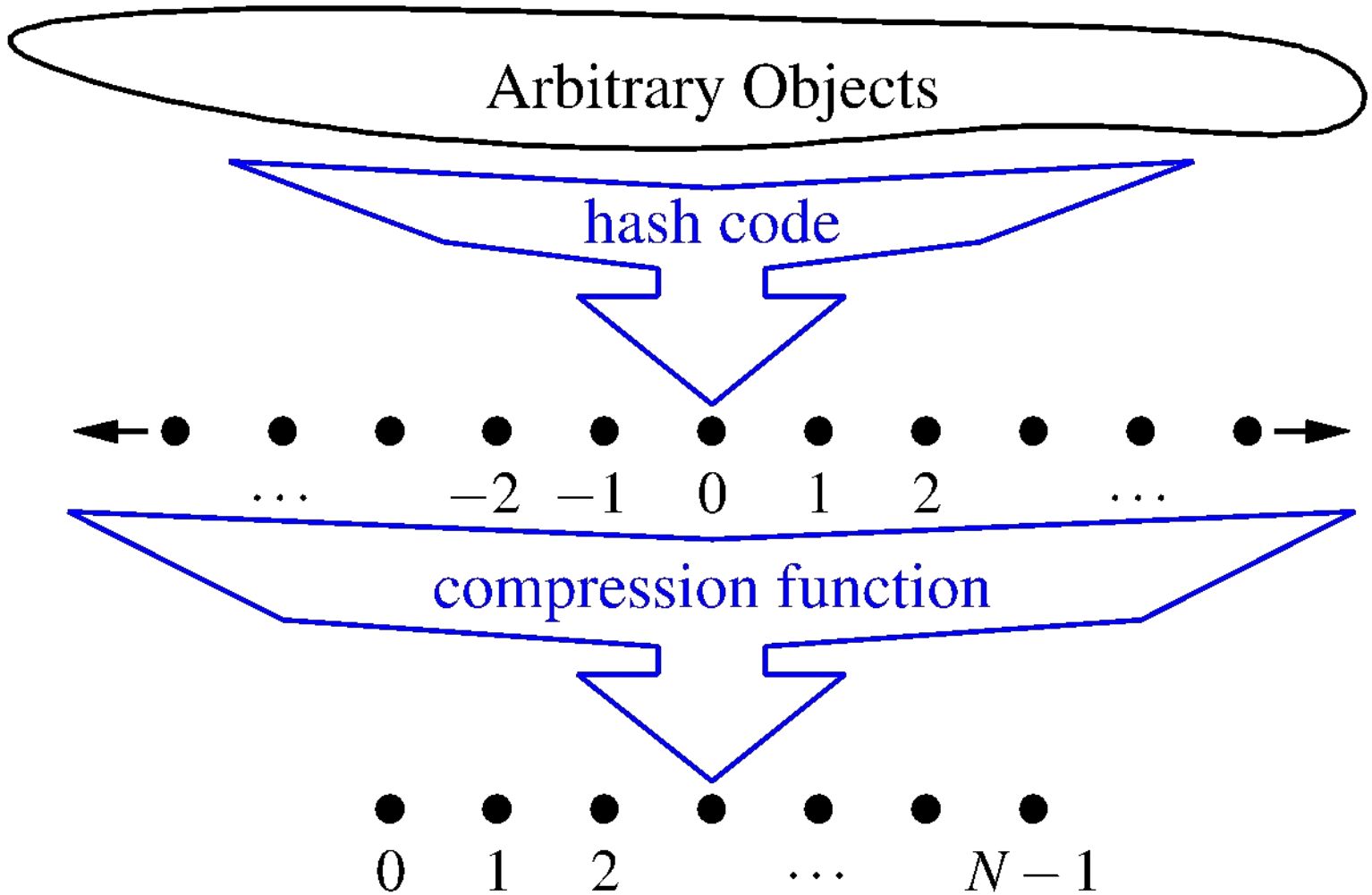
Hash Function Illustration



Hash Function Illustration



Hash Function Illustration



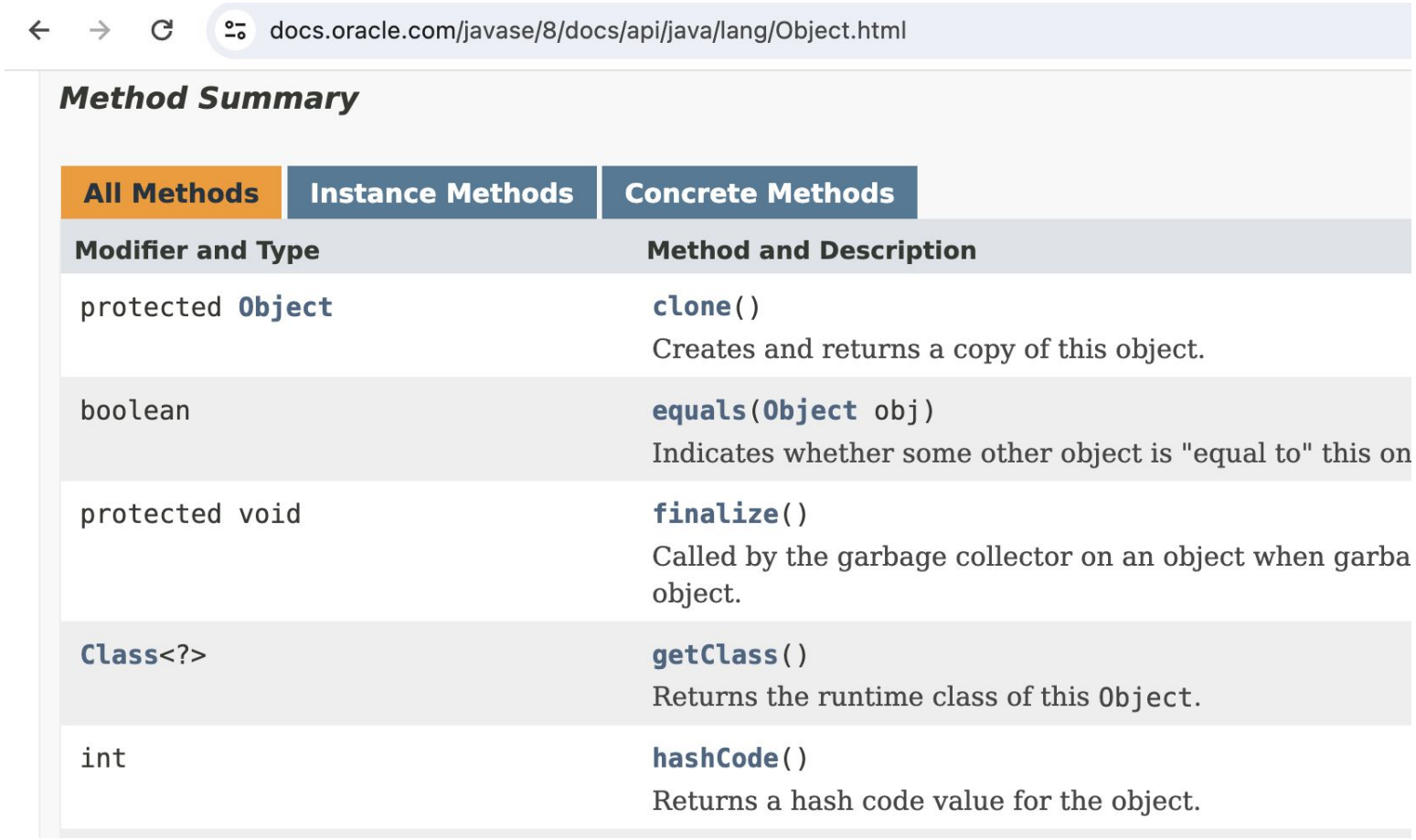
Hash Codes (h_1)

- $h_1(k)$ takes an arbitrary key and computes an integer
  - Goal: collision resistant!
  - Need not be a fixed length or in fixed range [0, N)
  - Can even be negative

Hash Codes h_1

Idea 1: Memory Addresses

- use the memory address where the keys are stored
- **default hash code for Java objects**



docs.oracle.com/javase/8/docs/api/java/lang/Object.html

Method Summary

All Methods | **Instance Methods** | **Concrete Methods**

Modifier and Type	Method and Description
protected **Object**	**clone**() Creates and returns a copy of this object.
boolean	**equals**(**Object** obj) Indicates whether some other object is "equal to" this on
protected void	**finalize**() Called by the garbage collector on an object when garba object.
Class<?>	**getClass**() Returns the runtime class of this **Object**.
int	**hashCode**() Returns a hash code value for the object.

Hash Codes (h_1)

What if our key is not an object?

- Integer cast: `byte`, `short`, `int`, `char` and `float`
- What about `long` and `double`??
  - Can't cast to int. We'll lose information!
  - COLLISIONS
  - Instead, partition bits into int components and combine them

Compression (h_2)

Why do we need compression?

Compression Idea 1: mod

$h_2(x) = x \bmod N$

forces output to be in range [0, N)

How should we choose N?

primes!

Compression (h_2)

Compression Idea 2: Multiply Add and Divide (MAD)

$h_2(x) = ((ax + b) \bmod p) \bmod N$

where N is the capacity
- p is a prime > N
- a and b are [0, p)
- a scales the range
- b shifts the start

HashMap

Book's `AbstractHashMap` hash method uses:

h_1`(k) = k.hashCode()` // java memory address

h_2`(x) = ((ax + b) % p) % N`

Hash Maps

Efficient data structure that stores (Key, Value) pairs

Implements the Map ADT

- `get(k)` : if the map `M` has an entry with key `k`, return its associated value; else, return null
- `put(k, v)` : insert entry `(k, v)` into the map `M`; if key `k` is not already in M, then return null; else, replace old value with `v` and return old value associated with `k`
- `remove(k)` : if the map `M` has an entry with key `k`, remove it from `M` and return its associated value; else, return null
- `size()`, `isEmpty()`
- `keySet()` : return an iterable collection of the keys in `M`
- `values()` : return an iterator of the values in `M`
- `entrySet()` : return an iterable collection of the entries in `M`

Hash Maps

Implementation

Let’s start with our ArrayMap and use hashes for indices

BE CAREFUL! % MEANS REMAINDER IN JAVA NOT MOD!

What should we do if there’s a collision?

- For a first impl, let’s just overwrite

Performance Analysis

	ArrayMap	Collision Resistant Hash Map
get		
put		
remove		

AbstractHashMap

```
1  public abstract class AbstractHashMap<K,V> extends AbstractMap<K,V> {
2    protected int n = 0;                    // number of entries in the dictionary
3    protected int capacity;                 // length of the table
4    private int prime;                      // prime factor
5    private long scale, shift;              // the shift and scaling factors
6    public AbstractHashMap(int cap, int p) {
7      prime = p;
8      capacity = cap;
9      Random rand = new Random( );
10     scale = rand.nextInt(prime−1) + 1;
11     shift = rand.nextInt(prime);
12     createTable( );
13   }
14   public AbstractHashMap(int cap) { this(cap, 109345121); }   // default prime
15   public AbstractHashMap( ) { this(17); }                     // default capacity
16   // public methods
17   public int size( ) { return n; }
18   public V get(K key) { return bucketGet(hashValue(key), key); }
19   public V remove(K key) { return bucketRemove(hashValue(key), key); }
20   public V put(K key, V value) {
21     V answer = bucketPut(hashValue(key), key, value);
22     if (n > capacity / 2)                 // keep load factor <= 0.5
23       resize(2 ∗ capacity − 1);           // (or find a nearby prime)
24     return answer;
25   }
```

AbstractHashMap

```
26    // private utilities
27    private int hashValue(K key) {
28      return (int) ((Math.abs(key.hashCode( )*scale + shift) % prime) % capacity);
29    }
30    private void resize(int newCap) {
31      ArrayList<Entry<K,V>> buffer = new ArrayList<>(n);
32      for (Entry<K,V> e : entrySet( ))
33        buffer.add(e);
34      capacity = newCap;
35      createTable( );                          // based on updated capacity
36      n = 0;                                   // will be recomputed while reinserting entries
37      for (Entry<K,V> e : buffer)
38        put(e.getKey( ), e.getValue( ));
39    }
40    // protected abstract methods to be implemented by subclasses
41    protected abstract void createTable( );
42    protected abstract V bucketGet(int h, K k);
43    protected abstract V bucketPut(int h, K k, V v);
44    protected abstract V bucketRemove(int h, K k);
45  }
```

Summary

1. Priority Queue
   a. removal order based on a “priority”
   b. efficiently implemented with a heap
2. Maps
   a. stores key-value pairs